Numerical Solution of the Robertson Kinetics IVP

Chuyang Wang, 2363591

Abstract

I treat the Robertson chemical-kinetics initial-value problem as a stiff three-species ODE on $0 \le t \le 40$. The vector field conserves mass, $y_1 + y_2 + y_3 = 1$, and the Jacobian always has a structural zero eigenvalue; the physically meaningful stiffness ratio, formed from the two nonzero eigenvalues, grows to $S(40) = 1.58 \times 10^5$. I implement explicit Euler, classical RK4, and implicit Euler with Newton's method, plus a step-doubling adaptive controller. Orders are measured against a closed-form logistic solution (Euler 0.943 ± 0.015, RK4 4.075 ± 0.018) and, on Robertson, against a tightened BDF/Radau oracle (implicit Euler 0.990 ± 0.002). Explicit Euler at $h = 0.01$ diverges, consistent with the frozen-Jacobian bound $h < 5.90 \times 10^{-4}$. Adaptive stepping does change $h(t)$, but at matched $y(40)$ error it spends more right-hand-side evaluations than a tuned fixed step, because each accepted step costs three implicit Euler solves. The main limitation is that $h\lambda$ overlays remain local diagnostics, not nonlinear stability proofs. A-stability of the time step, a small Newton residual, and a small mass defect are three different claims and are checked separately.

Keywords: implicit Euler, Newton iteration, stiffness, absolute stability, A-stability, step-doubling, Robertson problem

Contents

1 Introduction and Motivation **2**
- 1.1 Problem Background . . . 2
- 1.2 Mathematical Model . . . 2
- 1.3 Overview of the Report . . . 2

2 Discretisation and Analysis **3**
- 2.1 Discrete Schemes . . . 3
- 2.2 Consistency and Truncation Error . . . 3
- 2.3 Stability Analysis . . . 4
- 2.4 Convergence . . . 4

3 Implementation **4**
- 3.1 Algorithm . . . 4
- 3.2 Code Structure . . . 5
- 3.3 Key Design Decisions . . . 5

4 Numerical Experiments **5**
- 4.1 Verification: oracle and manufactured/exact cases . . . 5
- 4.2 Stability experiments . . . 10
- 4.3 Adaptivity . . . 13
- 4.4 Positivity (separate from conservation and from $E(h)$) . . . 14
- 4.5 Main results and cost–accuracy . . . 15
- 4.6 Limitations and robustness . . . 15

5 Conclusions **16**

A AI Transparency Log **17**
A.1 A1. Tools used 17
A.2 A2–A3. Claim audits 17
A.3 A4. Ownership 17

B Individual Contribution Statement **17**

C Supplementary data **18**

1 Introduction and Motivation

1.1 Problem Background

Robertson's 1966 three-species mechanism is the standard stiff test that exposes the gap between accuracy and absolute stability [1, 3]. The rate constants span eight orders of magnitude, so an explicit method that looks "accurate enough" on a smooth scalar problem still has to resolve the fastest Jacobian mode. I chose this topic (Problem Pack ③) because the model, the linear invariant, and the Week-2 scaffold already force the implicit-plus-Newton pathway the course asks me to own, rather than a conservative mechanics problem whose difficulty is geometric rather than stiff. This report is a solo submission: I derived the schemes, wrote the solvers, ran the experiments, and typeset the text.

1.2 Mathematical Model

The IVP is

$$\dot{y} = f(y), \qquad y(0) = (1, 0, 0)^\top, \qquad 0 \le t \le 40, \tag{1}$$

with mass-action kinetics (the factor of two in $B + B$ is already absorbed in the displayed $3 \times 10^7 y_2^2$ term [6])

$$\begin{aligned} \dot{y}_1 &= -0.04\, y_1 + 10^4 y_2 y_3, \\ \dot{y}_2 &= 0.04\, y_1 - 10^4 y_2 y_3 - 3 \times 10^7 y_2^2, \\ \dot{y}_3 &= 3 \times 10^7 y_2^2. \end{aligned} \tag{2}$$

The identity $1^\top f(y) = 0$ implies that every exact trajectory remains on the plane $y_1 + y_2 + y_3 = 1$. Consequently $J_f = \partial f / \partial y$ always has a zero eigenvalue. I define the stiffness ratio from the two nonzero eigenvalues,

$$S(t) = \frac{\max_{\lambda \neq 0} |\operatorname{Re} \lambda(t)|}{\min_{\lambda \neq 0} |\operatorname{Re} \lambda(t)|}, \tag{3}$$

using the numerical cutoff $|\lambda| < 10^{-8}$ to classify a computed eigenvalue as the structural zero. Along the BDF reference I built, $S(40) = 1.584 \times 10^5$ and $\lambda_{\min}(40) \approx -3.393 \times 10^3$.

1.3 Overview of the Report

Section 2 derives the three one-step methods, their stability functions, and the Newton equation. Section 3 records the implementation choices. Section 4 builds an independent oracle, measures orders with a fitted slope ±SE and a pointwise fitted-mean band (not a 95% confidence interval for p), tests the frozen-Jacobian stability prediction, reports conservation, positivity and full-state error separately, shows that the adaptive mesh actually moves with local error near tol, and compares cost at matched accuracy. Section 5 states what I trust and what I do not.

2 Discretisation and Analysis

2.1 Discrete Schemes

A one-step method is $y_{n+1} = y_n + h\,\Phi(t_n, y_n, h)$. Explicit Euler uses $\Phi = f(t_n, y_n)$:

$$y_{n+1} = y_n + hf(t_n, y_n). \tag{4}$$

Classical RK4 cancels Taylor terms through order four by four stages [4, 2]:

$$\begin{aligned} k_1 &= f(t_n, y_n), \\ k_2 &= f(t_n + h/2,\, y_n + (h/2)k_1), \\ k_3 &= f(t_n + h/2,\, y_n + (h/2)k_2), \\ k_4 &= f(t_n + h,\, y_n + hk_3), \\ y_{n+1} &= y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4). \end{aligned} \tag{5}$$

Implicit Euler evaluates the vector field at the unknown new state,

$$y_{n+1} = y_n + hf(t_{n+1}, y_{n+1}). \tag{6}$$

The unknown sits inside f, so each step is a nonlinear system

$$F(w) = w - y_n - hf(t_{n+1}, w) = 0, \qquad J_F(w) = I - hJ_f(t_{n+1}, w). \tag{7}$$

Newton's iteration is $J_F(w^{(k)})\delta = F(w^{(k)})$, $w \leftarrow w - \delta$, started from $w^{(0)} = y_n$, stopped at $\|F\|_\infty < 10^{-12}$.

2.2 Consistency and Truncation Error

If f is C^{p+1}, a method of order p has local defect $\mathcal{O}(h^{p+1})$ and, under a Lipschitz bound, global error $\mathcal{O}(h^p)$ on a fixed interval. Explicit and implicit Euler have $p = 1$; RK4 has $p = 4$. The step must also lie in the absolute-stability region: outside it the recurrence is not a perturbation of the ODE and a slope plot is meaningless.

The Tutorial 3 supplement records the implicit-Euler argument that I use here [10]. Write $u_n = y(t_n)$ for the exact IVP and y_n for the numerical values, with exact Newton solves and $e_n = y_n - u_n$, $e_0 = 0$. Taylor expansion of u *backwards* from t_{n+1} (where implicit Euler evaluates the slope) gives

$$u_{n+1} = u_n + hf(u_{n+1}) + d_{n+1}, \qquad d_{n+1} = -\tfrac{1}{2}h^2 y''(t_{n+1}) + \mathcal{O}(h^3). \tag{8}$$

Thus one exact step leaves an $\mathcal{O}(h^2)$ defect (local truncation $d_{n+1}/h = \mathcal{O}(h)$). Subtracting from $y_{n+1} = y_n + hf(y_{n+1})$ and using $\|f(a) - f(b)\| \le L\|a - b\|$ produces

$$(1 - hL)\|e_{n+1}\| \le \|e_n\| + Ch^2. \tag{9}$$

Over $N = T/h$ steps the defects accumulate; for $hL \le 1/2$ the amplification stays bounded by a constant depending only on L and T, so $\|e_N\| \le CTe^{2LT}h = \mathcal{O}(h)$. One-step $\mathcal{O}(h^2)$ times T/h steps is why the global order is one, not two. I do not claim this Lipschitz argument on the whole of $\mathbb{R}^3$; it holds in a neighbourhood of the mass simplex where the computed Robertson trajectory stays.

2.3 Stability Analysis

On the Dahlquist test equation $y' = \lambda y$ one has $y_{n+1} = R(z)y_n$ with $z = h\lambda$. The absolute-stability region is $S = \{z : |R(z)| \le 1\}$. Direct substitution gives

$$R_{\mathrm{E}}(z) = 1 + z, \qquad R_{\mathrm{RK4}}(z) = 1 + z + \frac{z^2}{2} + \frac{z^3}{6} + \frac{z^4}{24}, \qquad R_{\mathrm{IE}}(z) = \frac{1}{1 - z}. \tag{10}$$

A method is *A-stable* if $\{\mathrm{Re}\, z < 0\} \subset S$. Implicit Euler is A-stable (and L-stable, since $R(z) \to 0$ as $z \to \infty$). Explicit Euler's region is the disk of radius 1 centred at -1; RK4's region is larger but still bounded. For a real negative eigenvalue, explicit Euler therefore needs $0 < h \le 2/|\lambda|$.

At $t = 40$ on the oracle, $\lambda_{\mathrm{fast}} \approx -3.393 \times 10^3$, so the frozen-Jacobian explicit-Euler restriction is $h < 5.90 \times 10^{-4}$. Overlaying $z = h\lambda_j$ with the working implicit step $h = 0.1$ places the fast mode at $z \approx -339$, far to the left of both explicit regions and well inside the implicit half-plane. This overlay is a *local frozen-Jacobian diagnostic*, not a nonlinear stability theorem [3]: J_f varies along the trajectory, and non-normality can enlarge transients. I therefore also run an empirical step sweep (Section 4.2).

2.4 Convergence

On a Lipschitz IVP, consistency of order p plus zero-stability (satisfied by all three one-step methods) implies convergence of order p for sufficiently small h—the ODE form of Lax equivalence. Robertson's polynomial nonlinearity is locally Lipschitz, so the hypotheses hold in a neighbourhood of the positive mass plane. The numerical job is to check that the observed log-log slope matches p *after* h has entered the stability region.

3 Implementation

3.1 Algorithm

Algorithm 1 Implicit Euler step by Newton. Tolerance $\mathrm{tol} = 10^{-12}$.

procedure NewtonIE($f, J_f, y_n, t_{n+1}, h, \mathrm{tol}$)
  $w \leftarrow y_n$
  for iter $= 0, \ldots,$ maxiter **do**
    $F \leftarrow w - y_n - h \cdot f(t_{n+1}, w)$
    if $\|F\|_\infty < \mathrm{tol}$ **then**
      return w
    end if
    $J_F \leftarrow I - h \cdot J_f(t_{n+1}, w)$
    $\delta \leftarrow J_F^{-1} F$
    $w \leftarrow w - \delta$
  end for
end procedure

Algorithm 2 Step-doubling adaptive implicit Euler.

procedure AdaptiveIE($f, J_f, y_0, t_0, t_1, h_0, \text{tol}$)
  $t \leftarrow t_0, \quad y \leftarrow y_0, \quad h \leftarrow h_0$
  while $t < t_1$ **do**
    $h \leftarrow \min(h, t_1 - t, h_0)$
    $y_{\text{coarse}} \leftarrow$ one implicit Euler step of size h
    $y_{\text{fine}} \leftarrow$ two implicit Euler steps of size $h/2$
    $e \leftarrow \|y_{\text{fine}} - y_{\text{coarse}}\|_\infty$
    if $e \le \text{tol}$ **then**
      accept y_{fine}; $t \leftarrow t + h$
      if $e < \text{tol}/10$ **then** $h \leftarrow \min(2h, h_0)$
      end if
    else
      $h \leftarrow h/2$ ▷ reject; do not advance t
    end if
  end while
end procedure

3.2 Code Structure

File	Purpose
`code/model.py`	$f(t, y)$ and J_f for Robertson; logistic test problem
`code/methods.py`	Euler, RK4, implicit Euler+Newton, step-doubling
`code/plots.py`	log-log fits with `cov = True` and a fitted-mean ± 1.96 SE band
`code/run_all.py`	regenerates every report figure
`code/validation_tests.py`	PASS/FAIL: orders, mass, oracle, Jacobian
`.git/`	local history of iterative development (not a single commit)

The time-steppers take callables f and J_f; swapping the model does not touch the integrator. I implement Euler, RK4, and Newton myself. The only library solve of the IVP is the oracle `scipy.integrate.solve_ivp` (BDF/Radau), used as a reference, not as the method under test [5]. The submitted code archive includes the `.git` directory: `git log` records the model and Jacobian, then the time-steppers, then the experiment scripts and tests, then the figures. That history is the Dimension C record; there is no GitHub remote.

3.3 Key Design Decisions

Newton's initial guess is the previous step y_n, not an explicit Euler predictor. On the first Robertson step this guess is degenerate ($y_2 = 0$), so $\|F\|_\infty$ rises from 4.0×10^{-3} to 4.8×10^{1} before quadratic decay to 5.6×10^{-17} in twelve iterations—a feature of the basin, not a wrong Jacobian. The analytic Jacobian is cross-checked by central differences ($\max |J_{\text{an}} - J_{\text{FD}}| = 3.2 \times 10^{-12}$ at a generic state) and by the invariant test that every column of J_f sums to zero. The adaptive error signal is the method's own $O(h^2)$ difference $e = \|y_{\text{fine}} - y_{\text{coarse}}\|_\infty$; I accept y_{fine}.

4 Numerical Experiments

4.1 Verification: oracle and manufactured/exact cases

There is no closed form for (2). I built two independent tight-tolerance oracles,

$$\texttt{solve_ivp}(\cdot, \texttt{method} =' \texttt{BDF}', \texttt{rtol} = 10^{-12}), \qquad \texttt{method='Radau'}, \texttt{rtol} = 10^{-13},$$

and retained digits unchanged by that tightening. SciPy's `Radau` is three-stage Radau IIA (global order 5); one-stage Radau IIA is exactly implicit Euler [11]. I use the high-order code only as a checked reference, not as a method I claim to have implemented. At $t = 40$,

$$y_{\text{oracle}}(40) = (0.71582707,\ 9.185535 \times 10^{-6},\ 0.28416375), \tag{11}$$

with $\|y_{\text{BDF}} - y_{\text{Radau}}\|_\infty = 1.4 \times 10^{-11}$. This agrees with the problem-pack orientation value to all digits they print. These comparisons are empirical reference checks, not rigorous error bounds [10]. The logistic equation $y' = y(1 - y/10)$, $y(0) = 0.5$, supplies an exact scalar check for the explicit methods.

Logistic orders (Week-3 individual-challenge figure). Table 1 and Figure 1 report global error at $T = 10$. `np.polyfit`$(\log h, \log e, 1,$ `cov = True`$)$ gives the slope and one standard error. The shaded band is the pointwise fitted mean $\pm 1.96\,\mathrm{SE}(x)$ on a deterministic table: a fit diagnostic, not a 95% confidence interval for the order p, and not a Monte-Carlo interval.

Table 1: Global error at $T = 10$ on the logistic equation. Ratios are $e(h)/e(h/2)$.

h	Euler error	Euler ratio	RK4 error	RK4 ratio
0.40	5.787×10^{-3}	—	1.407×10^{-5}	—
0.20	3.203×10^{-3}	1.81	7.793×10^{-7}	18.1
0.10	1.668×10^{-3}	1.92	4.604×10^{-8}	16.9
0.05	8.490×10^{-4}	1.96	2.801×10^{-9}	16.4
0.025	4.281×10^{-4}	1.98	1.727×10^{-10}	16.2

Fitted slopes: Euler 0.943 ± 0.015 (theory 1), RK4 4.075 ± 0.018 (theory 4). The Euler slope sits slightly below 1 because the coarsest $h = 0.4$ is not yet in the asymptotic regime. There is no sharp numerical test for that regime [8]: in practice one reports that h has entered it once adjacent observed orders settle to a constant for sufficiently small h. Here the Euler ratios $1.81 \to 1.92 \to 1.96 \to 1.98$ do that. The shaded band on Figure 1 is that same fitted-mean diagnostic; for this noise-free error it is expected to be narrow [8]. RK4's successive ratios approach $16 = 2^4$.

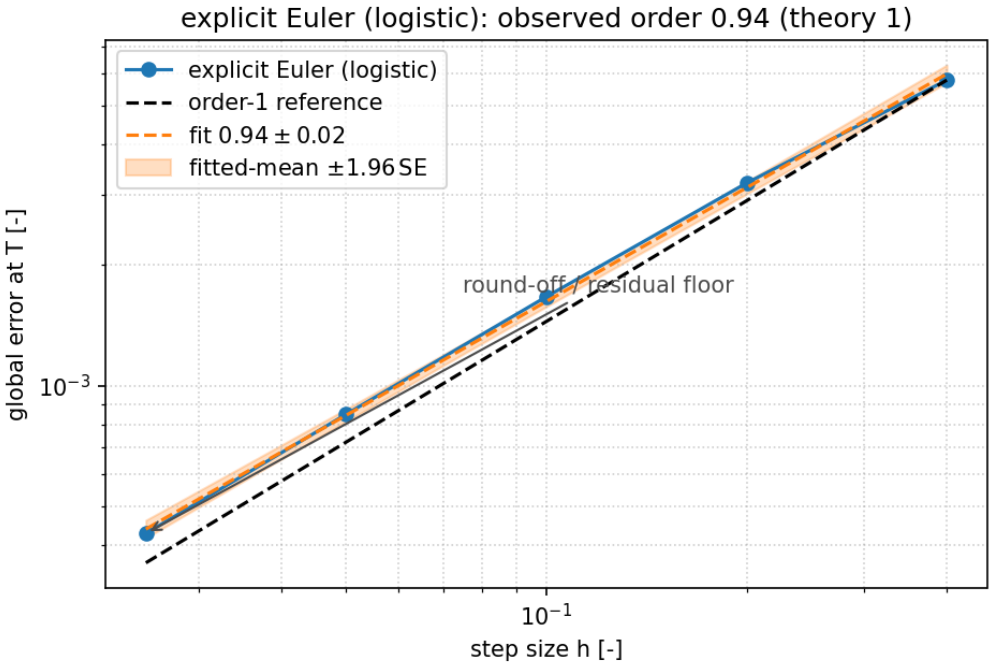


Figure 1: Week-3 figure. Explicit Euler on the logistic equation: log-log global error versus h, theoretical order-1 line, and OLS fit 0.943 ± 0.015. Following the Tutorial 3 supplement [10], $p \pm \mathrm{SE}$ is *one* standard error, not a 95% interval for p; the shaded region is a pointwise fitted-mean $\pm 1.96\,\mathrm{SE}$ diagnostic on a deterministic table (no Monte-Carlo noise). The arrow marks the smallest tabulated error, not a double-precision floor.

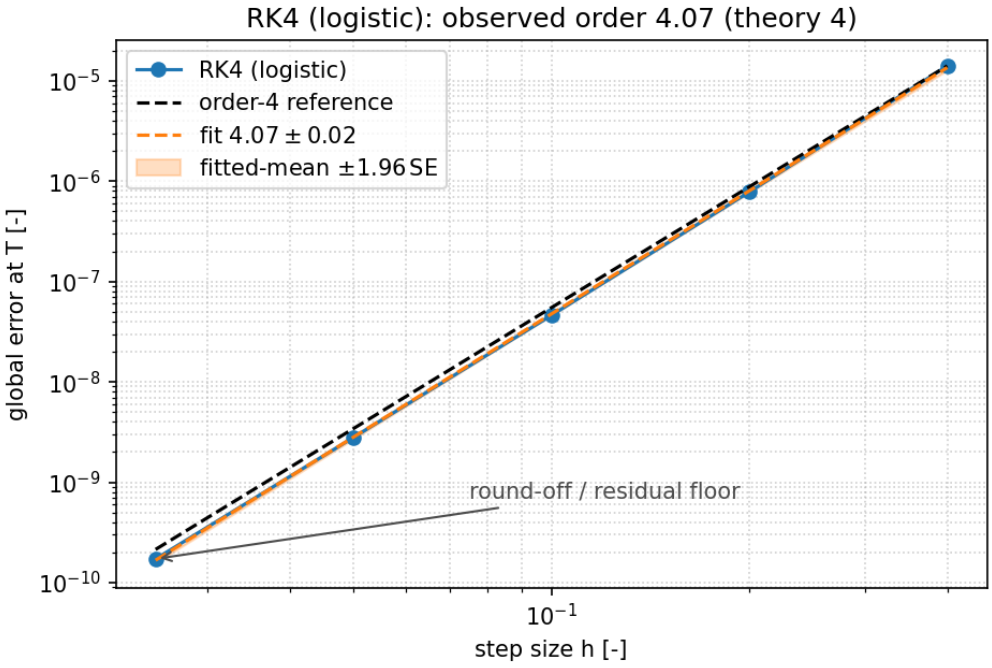


Figure 2: RK4 on the same logistic IVP. Observed order 4.075 ± 0.018 with the order-4 reference overlaid. This is the second required order measurement; it uses the closed-form solution, not the Robertson oracle.

Implicit Euler on Robertson. Against $y_{\mathrm{oracle}}(40)$, Table 3 and Figure 3 give

$$\text{slope } = 0.990 \pm 0.002.$$

At the working step $h = 0.1$,

$$y_{\mathrm{IE}}(40) = (0.716175,\ 9.199 \times 10^{-6},\ 0.283816),$$

with componentwise relative errors 0.049%, 0.147%, 0.122% (all $< 1\%$). Following the problem pack [6, 9] I keep three diagnostics apart: conservation, positivity, and full-state error. Mass defect $\max_n |1^\top y_n - 1| = 4.4 \times 10^{-16}$ at $h = 0.1$. After $t = 0$, $\min y_2 = 9.199 \times 10^{-6} > 0$ at the same h; Table 3 records $\min_{t>0} y_2$ for every tabulated step. Conservation alone is a weak test: an inaccurate or even negative vector can still sum to one. Accuracy is the oracle comparison $E(h)$, reported in the ∞-norm and also componentwise because y_2 is $\mathcal{O}(10^{-5})$.

Three different errors. Following the Tutorial 3 error budget [10] and the issue-ticket feedback [9, 8], write $\hat{y}_h$ for the computed implicit-Euler trajectory, y_h for the same method with exact Newton solves, y for the true IVP, and y_{ref} for the oracle. Then

$$\|\hat{y}_h - y_{\mathrm{ref}}\| \le \underbrace{\|y_h - y\|}_{\text{time discretisation}} + \underbrace{\|\hat{y}_h - y_h\|}_{\text{Newton}} + \underbrace{\|y - y_{\mathrm{ref}}\|}_{\text{reference}}. \tag{12}$$

A small Newton residual alone does not prove ODE accuracy. Time-step stability is $|R(h\lambda)| \le 1$ plus a step sweep; the nonlinear solve is evidenced by F and solver status; trajectory accuracy is $\|y_h - y_{\mathrm{ref}}\|$. Implicit Euler can be A-stable and still inaccurate: for $y' = -y$, $y(0) = 1$, $h = 1$ the step is stable and solved exactly, yet the error is 0.132. The residual is computable; $\|w - w_\star\|$ is only roughly bounded by $\|J_F^{-1}\|\,\|F\|$. On the first Robertson step the residual rises from 4.0×10^{-3} to 4.8×10^{1} before decaying: $y_2 = 0$ kills $\partial_y(3 \times 10^7 y_2^2)$, then the trial value activates it. That spike is not divergence. Solver convergence (Newton iterations staying small) and grid convergence ($E(h)$ still changing) answer different questions [10].

Newton tolerance versus time discretisation. Classroom feedback asked for a numerical check that the iteration error sits below the discretisation error, not only a residual sentence [9, 8]. Table 2 therefore freezes $h = 0.1$ and varies only the Newton stopping tolerance. Tightening from 10^{-8} to 10^{-12} changes the accepted $y(40)$ by

$$\|y(40; 10^{-8}) - y(40; 10^{-12})\|_\infty = 2.21 \times 10^{-7},$$

which is 6.3×10^{-4} times the time-discretisation error $E(0.1) = 3.48 \times 10^{-4}$ in Table 3. Even a deliberately loose tolerance 10^{-4} moves $y(40)$ by only 4.59×10^{-6}, still two orders below $E(0.1)$. I therefore keep $\|F\|_\infty < 10^{-12}$ as the production tolerance: the middle term of the error budget is negligible next to the $\mathcal{O}(h)$ column. Tightening the BDF/Radau oracle by $100\times$ changed $y(40)$ by only 1.4×10^{-11}, so the reference term is smaller still.

Table 2: Newton tolerance sweep at fixed $h = 0.1$. Δy is the ∞-norm difference against the 10^{-12} run; E is the same comparison against the BDF oracle. Iteration error is the Δy column; discretisation error is the E column.

Newton tol	$\|y(40) - y(40; 10^{-12})\|_\infty$	$E(0.1)$ vs oracle	RHS evaluations
10^{-4}	4.590×10^{-6}	3.525×10^{-4}	809
10^{-6}	3.151×10^{-6}	3.511×10^{-4}	822
10^{-8}	2.209×10^{-7}	3.481×10^{-4}	1023
10^{-10}	3.670×10^{-12}	3.479×10^{-4}	1210
10^{-12}	0	3.479×10^{-4}	1257

Table 3: Implicit Euler on Robertson. Full-state error $E(h) = \|y(40) - y_{\text{oracle}}\|_\infty$ is separate from positivity $\min_{t>0} y_2$ (the IVP has $y_2(0) = 0$). Adjacent observed order $p_{\text{pair}} = \log(E(h)/E(h/2))/\log 2$ [10]. Cost is RHS evaluations; Jacobian factorisation is not counted. Newton converges on every row, while $E(h)$ still halves: solver convergence is not grid convergence.

h	$E(h)$	ratio	p_{pair}	$\min_{t>0} y_2$	RHS
0.80	2.711×10^{-3}	—	—	9.292×10^{-6}	213
0.40	1.375×10^{-3}	1.97	0.98	9.239×10^{-6}	378
0.20	6.930×10^{-4}	1.98	0.99	9.213×10^{-6}	671
0.10	3.479×10^{-4}	1.99	0.99	9.199×10^{-6}	1257
0.05	1.743×10^{-4}	2.00	1.00	9.192×10^{-6}	2424

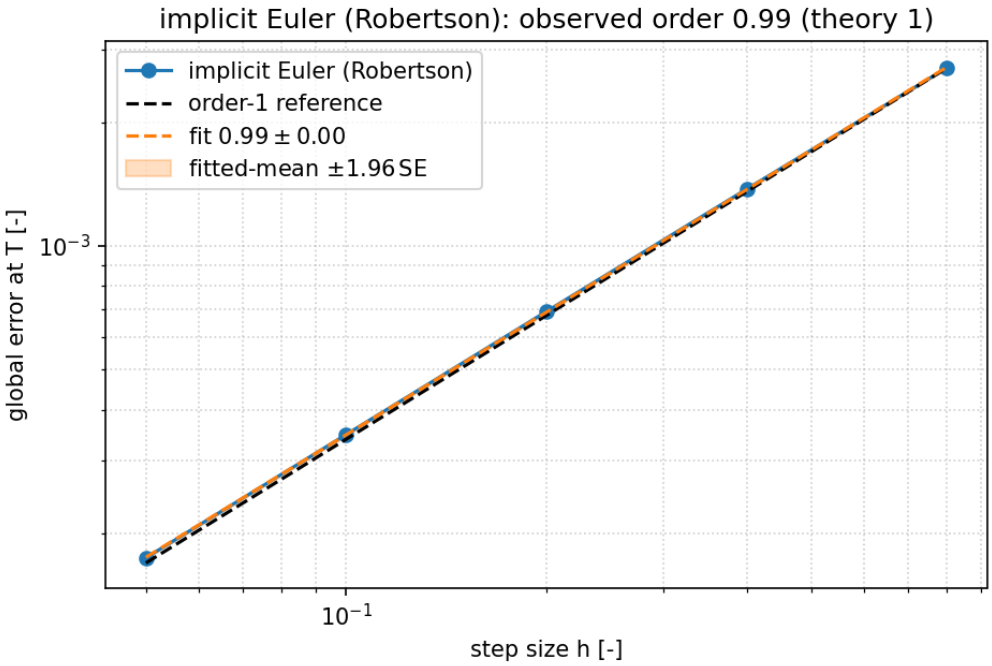


Figure 3: Implicit Euler on Robertson versus the BDF/Radau oracle. OLS slope 0.990 ± 0.002 (one standard error). The band is a deterministic fit diagnostic; p_{pair} in Table 3 has already settled near 1, which is the practical test for the asymptotic regime.

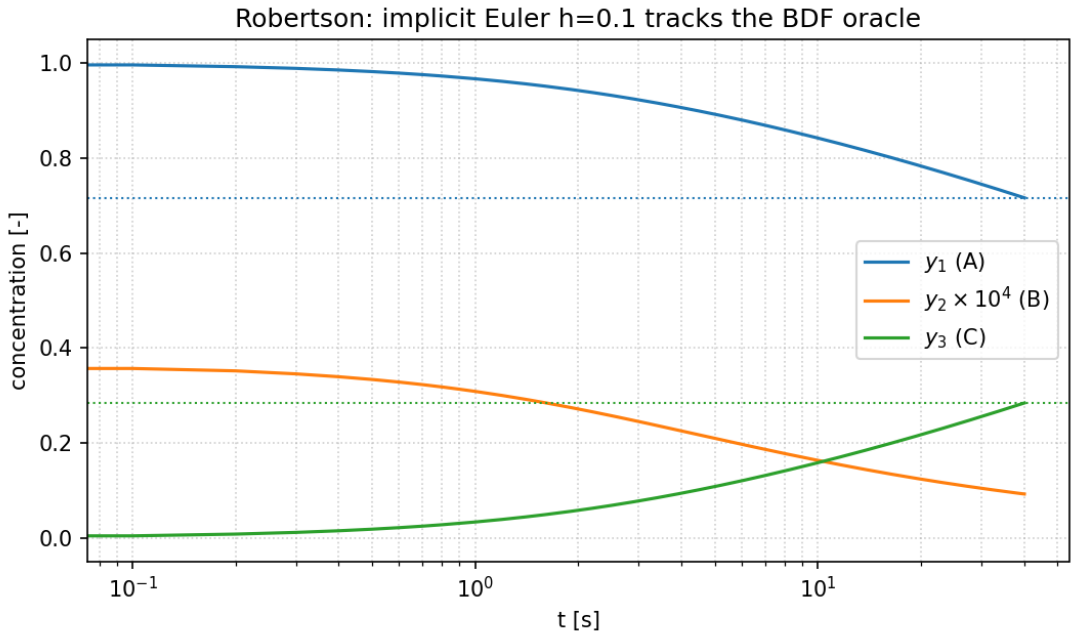


Figure 4: Robertson trajectory with implicit Euler, $h = 0.1$, logarithmic time. y_2 is scaled by 10^4. Dotted lines mark the oracle terminal values of y_1 and y_3. The plot shows that a stable first-order method tracks the slow drift; it does not by itself prove the order (that is Figure 3).

4.2 Stability experiments

Figure 5 shows the three real parts of $\mathrm{eig}(J_f(y(t)))$ along the oracle. One eigenvalue stays at machine zero; one slow mode remains $O(10^{-2})$; the fast mode grows to -3.39×10^3. Figure 6 plots $S(t)$ from the first positive logarithmic time (the initial state $y_2 = 0$ is degenerate). At $t = 40$, $S = 1.584 \times 10^5$.

The frozen-Jacobian bound for explicit Euler is $h < 5.90 \times 10^{-4}$, i.e. $\sim 6.8 \times 10^4$ steps on $[0, 40]$. Running $h = 0.01$ (only 4×10^3 steps) already produces inf/NaN (Figure 8). This is a stability failure, not an accuracy failure: the true solution stays on the mass simplex. Figure 7 overlays $h\lambda_j$ at $t = 40$ with $h = 0.1$. The fast mode lies far outside both explicit regions and inside the implicit half-plane, matching the blow-up experiment. I do not claim this overlay proves nonlinear stability; the experiment does.

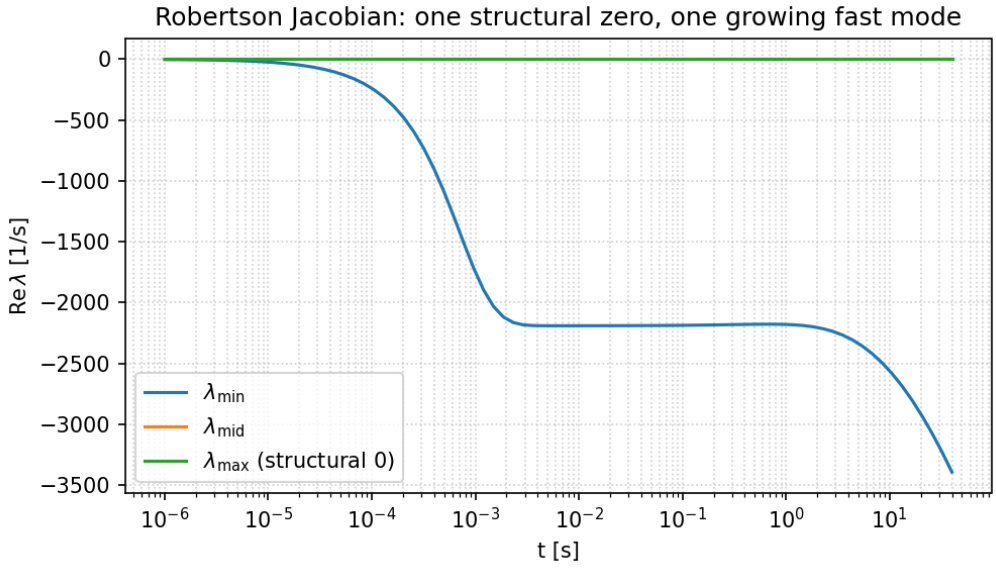


Figure 5: Real parts of the Robertson Jacobian along the BDF interpolant. The structural zero is a conservation identity, not a dynamical mode, and is excluded from $S(t)$.

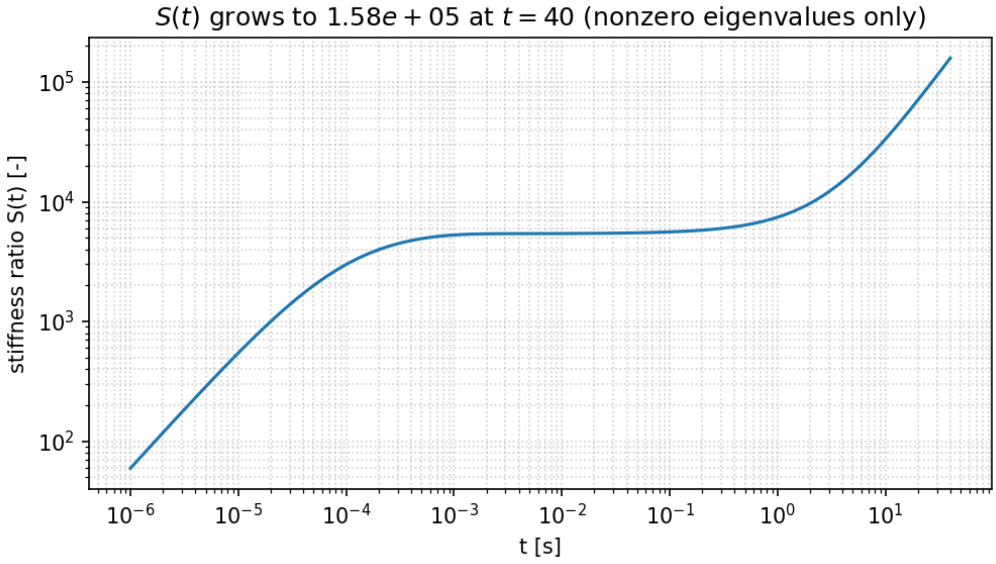


Figure 6: Nonzero-eigenvalue stiffness ratio versus logarithmic time. $S(40) = 1.58 \times 10^5$ is computed from the oracle I built, not copied from the problem pack.

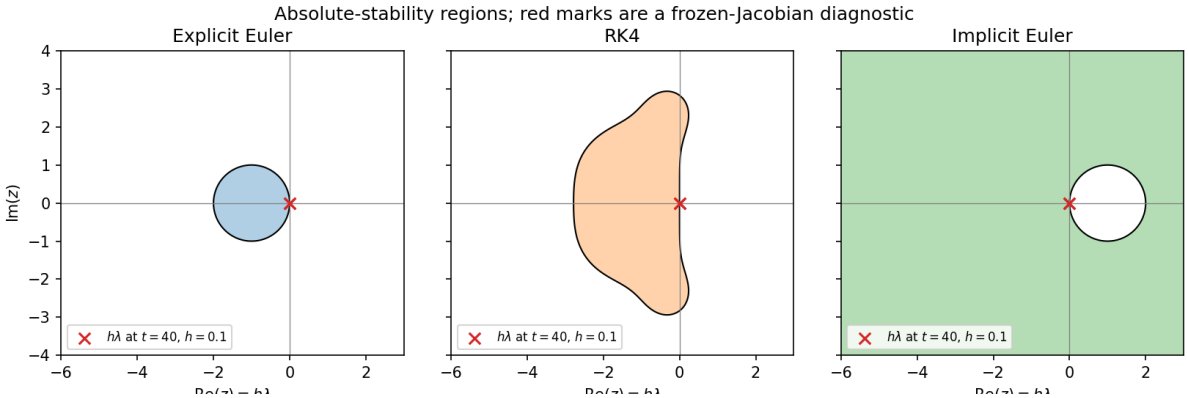


Figure 7: Absolute-stability regions of Euler, RK4, and implicit Euler. Red crosses: $h\lambda_j$ at $t = 40$ with the implicit working step $h = 0.1$. Points outside an explicit region are a warning, confirmed by Figure 8; points inside the implicit region are not a nonlinear proof.

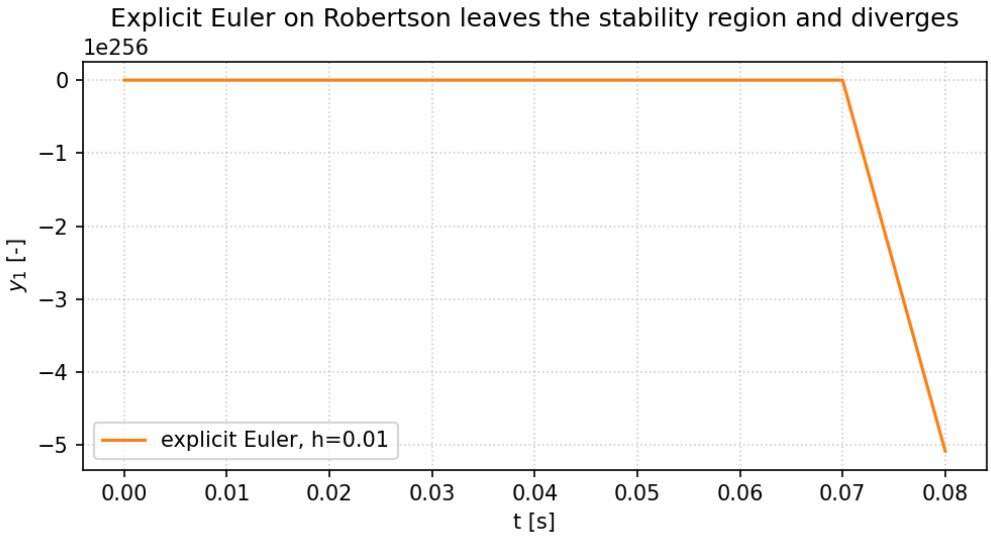


Figure 8: Explicit Euler, $h = 0.01$, $T = 40$: the trajectory leaves the floating-point range. The same h is two orders of magnitude larger than $2/|\lambda|_{\max}$. This is the stiff failure mode, not a large truncation error on a bounded orbit.

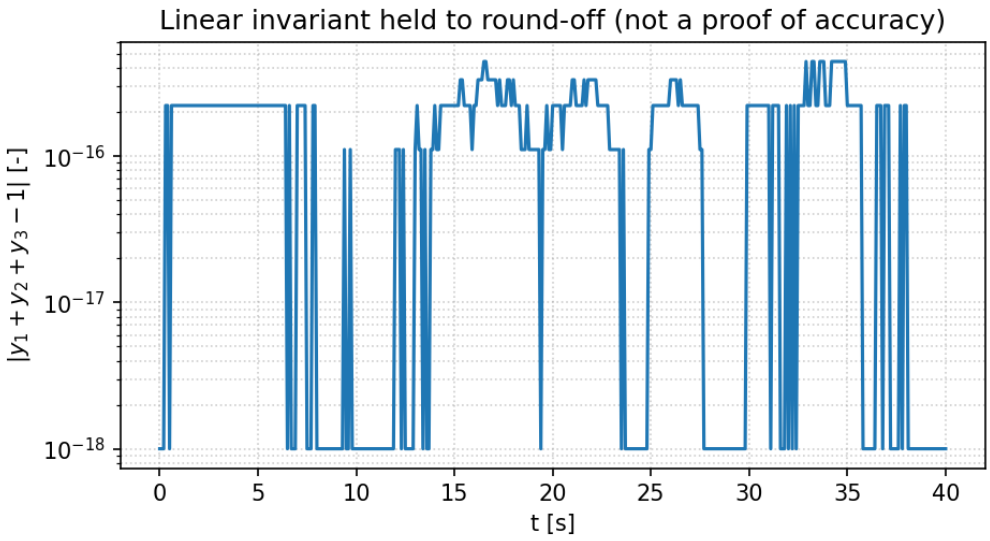


Figure 9: Mass defect of implicit Euler, $h = 0.1$. The defect sits at round-off, as expected for a linear invariant when the stage equation is solved tightly. An inaccurate solution can still conserve the sum; compare Figure 3 before claiming correctness.

4.3 Adaptivity

With $h_0 = 0.1$ and $\mathrm{tol} = 2 \times 10^{-6}$, the controller accepts 749 steps, rejects 3, reaches a smallest $h = 1.25 \times 10^{-2}$, and returns to $h = h_0$ by the end of the interval (Figure 10, top). Terminal error versus the oracle is $e_\infty = 8.90 \times 10^{-5}$, smaller than the fixed-$h = 0.1$ error 3.48×10^{-4}. The step size *does* change; a relabelled fixed grid would not produce this $h(t)$.

The brief also asks whether the local error sits near the tolerance, not only whether h varies [6]. Figure 10 (bottom) plots the accepted estimator $e(t) = \|y_{\mathrm{fine}} - y_{\mathrm{coarse}}\|_\infty$. Every accepted e lies at or below $\mathrm{tol} = 2 \times 10^{-6}$ (maximum 1.13×10^{-6}); the median is 3.22×10^{-7}; and 89.2% of accepted steps fall in the keep-h band $[\mathrm{tol}/10, \mathrm{tol}]$. The three rejected attempts are exactly the controller's response when e would have exceeded tol. So h moves, and the local error tracks the tolerance rather than collapsing far below it.

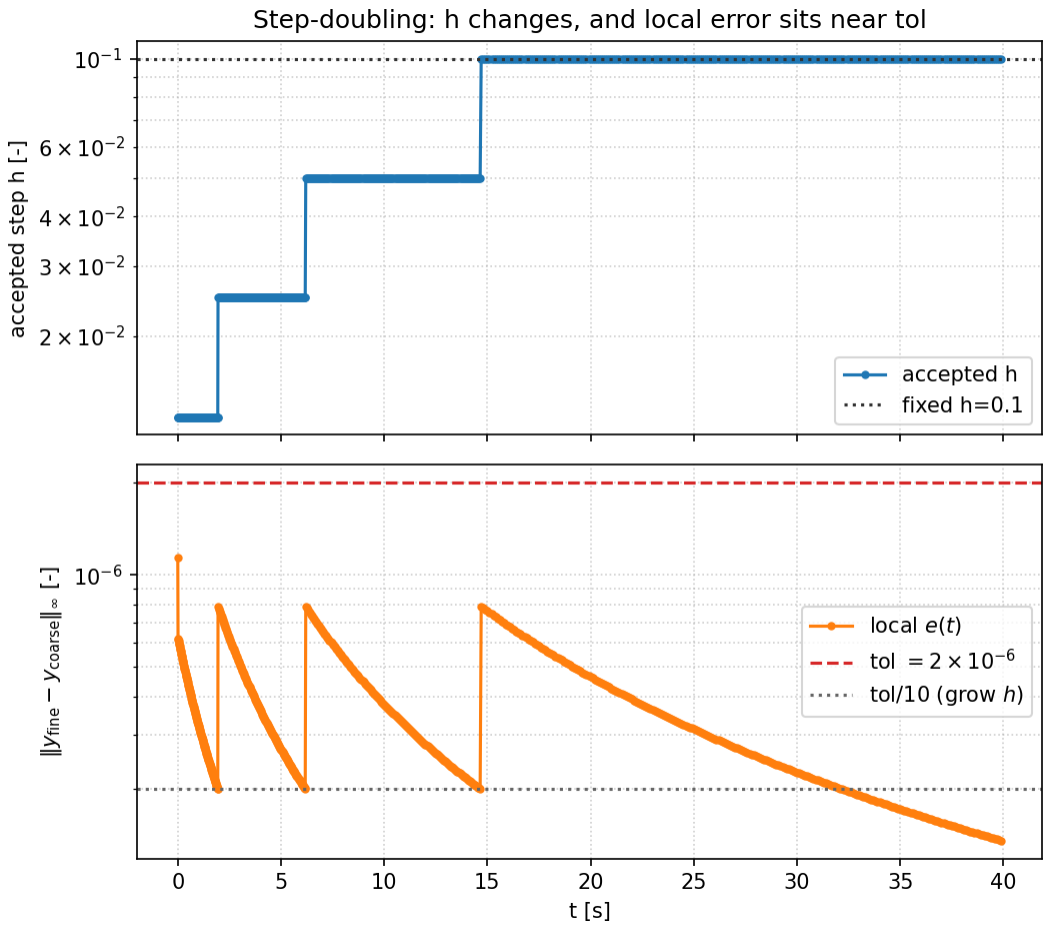


Figure 10: Step-doubling implicit Euler ($h_0 = 0.1$, tol $= 2 \times 10^{-6}$). Top: accepted $h(t)$ shrinks through the early transient and grows back to h_0. Bottom: accepted local error $e(t)$ stays at or below tol and mostly in $[\text{tol}/10, \text{tol}]$. 749 steps accepted, 3 rejected.

4.4 Positivity (separate from conservation and from $E(h)$)

Robertson concentrations are physically nonnegative. Implicit Euler has no positivity theorem: a large implicit step on other kinetics can cross through zero while still conserving $1^\top y$. I therefore measured $\min_{t>0} y_2$ on the raw method (no limiter) for $h \in \{0.05, 0.1, 0.2, 0.4, 0.8, 2, 5, 10\}$. After the initial state $y_2(0) = 0$, every accepted value stayed positive; the smallest post-initial entry at the working step $h = 0.1$ is 9.199×10^{-6}, and even at $h = 10$ one has $\min_{t>0} y_2 = 1.035 \times 10^{-5}$ with $n_{\text{neg}} = 0$. Newton trial states on the first step also stayed at or above zero (the minimum iterate is the initial guess $w_2 = 0$).

I still coded two empirical repairs, and I state the limitation in the same sentence: **neither has a convergence guarantee**.

- *Clip:* replace a negative component by 0 after the Newton update. This does not restore mass, so it can break the linear invariant that implicit Euler otherwise holds to round-off.
- *Reject:* abort the step if the accepted state or any Newton iterate is negative.

On this IVP both rules are no-ops: clip never changes a component, and reject never fires, through $h = 10$. That is an observation, not a proof. A write-up that treated clip as a positivity-preserving scheme would be claiming a theorem the method does not have. I therefore leave the production integrator unclipped, report $\min y_2$ in Table 3, and keep positivity out of the $E(h)$ column.

4.5 Main results and cost–accuracy

The central question is whether implicit Euler is worth its Newton cost on this stiff IVP. I name the cost measure first: **right-hand-side evaluations** accumulated during Newton residual computations. I do *not* count Jacobian assembly, dense linear solves, Python overhead, or plotting, and I hold those exclusions fixed across methods.

Figure 11 plots $e_\infty(40)$ against that cost. Fixed-step implicit Euler at $h = 0.05$ reaches $e_\infty = 1.74 \times 10^{-4}$ with 2424 RHS calls. Adaptive stepping at $\mathrm{tol} = 5 \times 10^{-5}$ reaches the same error with 3680 RHS calls (400 accepted steps). Tighter adaptive tolerances spend even more: $\mathrm{tol} = 2 \times 10^{-6}$ uses 6835 evaluations for $e_\infty = 8.90 \times 10^{-5}$. The method I expected to win on work *did not*: each accepted adaptive step costs three implicit Euler solves, so step-doubling is a reliability mechanism, not a cheaper integrator, once a good fixed h is already known. Wall-clock on this machine was 0.03 s (fixed $h = 0.05$) versus 0.04 s (adaptive, same error); I do not treat wall-clock as the primary cost because it is hardware-dependent.

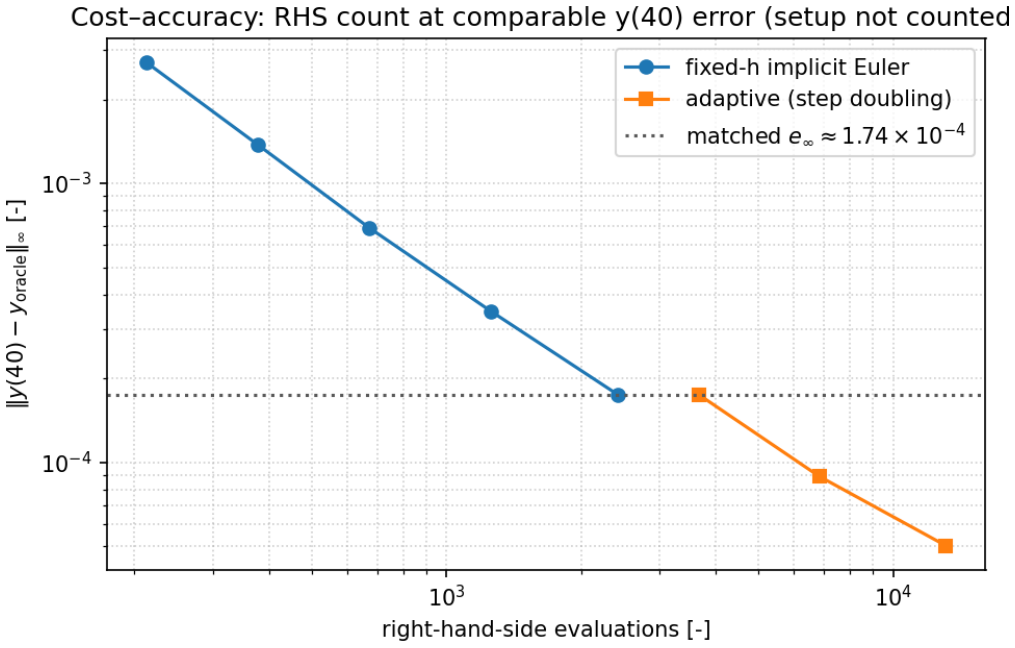


Figure 11: Learning-outcome D. Error at $t = 40$ versus RHS evaluations, matched against the same BDF oracle. The dotted line marks the matched accuracy $e_\infty \approx 1.74 \times 10^{-4}$. Adaptive step-doubling lies to the right of the fixed-h curve: more work at a given accuracy. The conclusion is reported because the measurement went that way, not because I hoped it would.

4.6 Limitations and robustness

The $h\lambda$ picture is frozen at one state. During the first 10^{-4} time units the Jacobian is nearly degenerate and Newton residuals rise before they fall; a code that stops on residual increase would abort a correct solve. Section 4.4 shows that y_2 did not go negative on this IVP, even at $h = 10$; that observation is not a positivity theorem, and the clip/reject repairs I tried have no convergence guarantee. Step-doubling assumes a smooth error expansion $\tau \approx Ch^2$, which holds here but would fail across a non-smooth right-hand side. I did not implement a positivity-preserving scheme or a Rosenbrock method; those remain the natural extensions.

5 Conclusions

- I solved Robertson's IVP with three one-step methods, an analytic Newton Jacobian, and a step-doubling controller, all checked against a BDF/Radau oracle that I tightened myself.
- Observed orders match theory once h is in the asymptotic regime: Euler 0.943 ± 0.015, RK4 4.075 ± 0.018, implicit Euler on Robertson 0.990 ± 0.002.
- Conservation, positivity and full-state error are separate: mass sits at round-off, $\min_{t>0} y_2 > 0$ on every tabulated h, and $E(h)$ is still $\mathcal{O}(h)$. Clip and reject never fired; I do not claim a positivity theorem.
- At $h = 0.1$, $\|y(40; 10^{-8}) - y(40; 10^{-12})\|_\infty = 2.21 \times 10^{-7}$ is far below the discretisation error 3.48×10^{-4}.
- Adaptive stepping accepts 749 steps, rejects 3, and keeps 89% of local errors in $[\text{tol}/10, \text{tol}]$. At matched $y(40)$ error it still spends more RHS evaluations than a tuned fixed step.
- A-stability, a small Newton residual, and mass conservation are not accuracy claims; the evidence for accuracy is the refined oracle comparison.

References

[1] H. H. Robertson, "The solution of a set of reaction rate equations," in *Numerical Analysis: An Introduction*, J. Walsh, Ed. London: Academic Press, 1966.

[2] E. Hairer, S. P. Nørsett and G. Wanner, *Solving Ordinary Differential Equations I*, 2nd ed. Berlin: Springer, 1993.

[3] E. Hairer and G. Wanner, *Solving Ordinary Differential Equations II*, 2nd ed. Berlin: Springer, 1996.

[4] J. C. Butcher, *Numerical Methods for Ordinary Differential Equations*, 3rd ed. Chichester: Wiley, 2016.

[5] P. Virtanen et al., "SciPy 1.0: fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

[6] Course problem pack, "Chemical kinetics: Robertson problem," Project 1, Numerical Analysis of ODEs and PDEs, 2026.

[7] C. R. Harris et al., "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.

[8] H. Zhang, "Feedback on Issue Tickets," MTH321 forum announcement, 27 September 2026.

[9] H. Zhang, "Classroom feedback: recurring difficulties" (Week 2 issue tickets), course slides, September 2026.

[10] Course handout, "Tutorial 3: scientific visualization" (supplement), September 2026.

[11] Course handout, "Radau methods for ODE systems," September 2026.

A AI Transparency Log

A.1 A1. Tools used

Tool	Date	Primary use
Cursor (Grok 4.6 agent)	15–21 Sep 2026	Newton/implicit Euler in `methods.py`, first experiment scripts
Cursor (Grok 4.6 agent)	27 Sep 2026	`run_all.py` and `validation_tests.py` refinements
Cursor (Grok 4.6 agent)	5 Oct 2026	positivity probe, Newton-tolerance sweep, adaptive $e(t)$ tracking

A.2 A2–A3. Claim audits

Item 1: analytic Robertson Jacobian. Prompt/claim: fill J_f from the three-term RHS, including the $6 \times 10^7 y_2$ off-diagonals. Risk: a diagonal-only Jacobian drops the chemistry and still produces a "running" solver. Independent check: central finite differences at $y = (0.9, 10^{-5}, 0.1)$, $\max |J_{\text{an}} - J_{\text{FD}}| = 3.2 \times 10^{-12}$; column sums of J_f equal 0 to machine precision (`run_all.py` log; Figure 9 is the integrated counterpart). Decision: keep the analytic Jacobian; reject any AI/edit that fills only the diagonal.

Item 2: adaptive step-doubling cost. Prompt/claim: "adaptive stepping should be cheaper than a fixed grid." Risk: counting accepted steps instead of RHS evaluations hides the three-solve estimator. Independent check: Figure 11 and the printed lines `n_rhs=2424` (fixed $h = 0.05$) versus `n_rhs=3680` (adaptive, same e_∞). Decision: report the negative result; name the cost as RHS evaluations; do not claim a speed-up.

Item 3: first Newton residual rise. Prompt/claim: a rising residual means a wrong J_F or a failed solve. Risk: "debugging" a correct basin-of-attraction effect. Independent check: first-step trace at $t = 0.1$, $h = 0.1$: $\|F\|_\infty$ sequence 4.0×10^{-3}, 4.8×10^{1}, …, 5.6×10^{-17}, matching the Week-2 scaffold comment that $y_2 = 0$ is degenerate. Decision: keep the undamped Newton start; document the spike as expected.

A.3 A4. Ownership

I own the mathematical write-up, the figures, and the interpretation. AI assistance covered filling and checking `robertson_rhs`/`robertson_jacobian`, Euler/RK4/Newton/step-doubling, the oracle and positivity/Newton-tolerance/adaptive sweeps in `run_all.py`, and `validation_tests.py`. Trusted dependency: `numpy.linalg.solve` for the Newton correction; `scipy.integrate.solve_ivp` only as an independent reference (BDF/Radau, stated tolerances). I can explain and modify these methods without AI during the viva.

B Individual Contribution Statement

This is a solo project, so there is one ICS.

Individual Contribution Statement — Project 1 (ODE IVP)

Name: Chuyang Wang Student number: 2363591

Role this project: sole author (implementation, weekly tasks, report).

The specific things I did this project:

1. Implemented `robertson_rhs`, `robertson_jacobian`, Euler, RK4, `newton_implicit_euler` and `adaptive_implicit_euler` in `code/model.py` and `code/methods.py`.

2. Completed the weekly issue tickets and individual challenges, including the Week-2 RC-diode $h_{\max}$ pair (0.2 ms $\to$ 20 ms) and the Week-3 before/after log-log convergence figure (slope 0.943 ± 0.015).

3. Built the BDF/Radau oracle, wrote `validation_tests.py`, and regenerated all report figures with `python code/run_all.py`.

4. Added the positivity probe (clip / reject, no convergence claim), the Newton-tolerance sweep at $h = 0.1$, and the adaptive $e(t)$ panel with accept/reject counts.

5. Wrote the full LaTeX report: model, stability, Newton analysis, experiments, cost–accuracy comparison, AI Transparency Log and this ICS.

What I would do differently: freeze the cost measure (RHS evaluations) before plotting Figure 11, instead of discovering afterwards that adaptive stepping is more expensive than a tuned fixed step. Anything I should have asked for help with earlier: whether a rising Newton residual on the first Robertson step ($y_2 = 0$) is expected, before treating it as a Jacobian bug. Signature: _____ Date: _____

C Supplementary data

Newton residuals on the first Robertson step ($t = 0.1$, $h = 0.1$), $\|F\|_\infty$:

$$4.00 \times 10^{-3},\ 4.76 \times 10^{1},\ 1.19 \times 10^{1},\ 2.98,\ 7.43 \times 10^{-1},\ \ldots,\ 5.55 \times 10^{-17} \quad \text{(12 iterations)}.$$

Reproduced by `python code/run_all.py` (block `[newton]`).